GUÍA DE SUSTENTACIÓN ORAL • ASIGNATURA DE ASEGURAMIENTO DE LA CALIDAD (QA)

Guion Práctico para la Exposición del Proyecto de Pruebas

Sistema de Facturación Electrónica UBL 2.1 (SUNAT) • Suite PHPUnit 10.5 (39 Tests / 140 Assertions)

Información del Alumno y el Proyecto:

Rol: Estudiante de Sistemas / QA Engineer

Asignatura: Testing y Calidad de Software

Duración estimada: 7 a 10 minutos

Total de Tests a sustentar: 39 pruebas (100% aprobadas)

Estructura Recomendada para tu Exposición:

Paso	Etapa	¿Qué vas a mostrar?	Tiempo
Paso 1	Introducción	Problema real de facturación y por qué se necesita QA	1 min
Paso 2	Terminal CLI	Correr los 39 tests de PHPUnit en 0.21 segundos	1.5 min
Paso 3	Código Unitario	VS Code: Algoritmo RUC Módulo 11 y Factura Mixta (IGV)	2 min
Paso 4	Integración	VS Code: Detección de Adulteración (2335) e Idempotencia	2 min
Paso 5	Mocks de SUNAT	VS Code: Simulación de caída de red y Rollback de stock	1.5 min
Paso 6	Sistema Web	Navegador: Emitir factura en vivo, ver XML y consola de tests	1.5 min

Consejo para la exposición: No leas el código línea por línea. El profesor ya sabe programar; lo que quiere escuchar es *qué técnica de QA usaste, por qué elegiste ese caso de prueba y cómo tu prueba salvó a la empresa de un error grave o una multa*.

PASO 1 Introducción y Contexto del Negocio (1 minuto)

QUÉ TENER ABIERTO EN LA PANTALLA:

Ten abierto tu navegador web en `http://localhost:5173` mostrando la pantalla de emisión, o ten abierta la portada del PDF.

QUÉ DECIR CON TUS PROPIAS PALABRAS:

*"Buenos días profesor y compañeros. Hoy les voy a presentar la estrategia de pruebas y aseguramiento de la calidad (QA) aplicada a un **Sistema de Facturación Electrónica bajo la normativa UBL 2.1 de SUNAT**.*

*En la vida real, un sistema de facturación no puede tener fallas: si calculamos mal un impuesto por un solo céntimo, si dejamos que una factura se envíe dos veces por error de la cajera, o si alteran el archivo XML, la empresa recibe una multa tributaria inmediata. Por eso, diseñamos una suite automatizada con **PHPUnit 10** que prueba absolutamente todo antes de tocar un ambiente real."*

JUSTIFICACIÓN SENCILLA (POR QUÉ LO HICISTE):

Estableces el problema del mundo real. Le demuestras al profesor que entiendes que el testing no es solo 'ver que compile', sino proteger el dinero y la legalidad del negocio.

PASO 2 Demostración en Terminal - Corriendo los 39 Tests (1.5 minutos)

QUÉ HACER EN LA COMPUTADORA:

1. Abre la terminal (PowerShell o Git Bash).
2. Ubícate en la carpeta del backend: `cd Billing/backend`
3. Escribe y ejecuta: `.\vendor\bin\phpunit --testdox --colors=never`

QUÉ DECIR MIENTRAS CORRE LA TERMINAL:

"Para empezar, quiero que vean las pruebas ejecutándose en vivo. Voy a correr el comando oficial de PHPUnit con el parámetro `--testdox`, que nos describe cada prueba en lenguaje humano.

(Señalas la pantalla cuando terminen los checks verdes)
*Como pueden ver, se ejecutaron **39 pruebas y 140 aserciones en apenas 0.21 segundos**. El resultado es 100% aprobado con cero errores.*

*¿Por qué corren tan rápido? Porque aplicamos los **principios FIRST** de QA: todas las pruebas son independientes y corren en memoria RAM, sin depender de servidores externos lentos ni de bases de datos pesadas."*

CONCEPTO CLAVE DE QA A RESALTAR:

Principio FIRST (Fast): Las pruebas deben ser ultrarrápidas para que los desarrolladores puedan ejecutarlas decenas de veces al día sin perder tiempo.

PASO 3 Inspección en Visual Studio Code: Pruebas Unitarias (2 minutos)

QUÉ CARPETA Y ARCHIVOS ABRIR EN VS CODE:

1. Carpeta: `backend/tests/Unit/` → Abre `ValueObjectsTest.php`
2. Carpeta: `backend/src/Domain/ValueObjects/` → Abre `Ruc.php` (líneas 24 a 37)
3. Carpeta: `backend/tests/Unit/` → Abre `CalculadoraTributariaTest.php` (líneas 44 a 104)

QUÉ DECIR EXPLICANDO EL RUC Y EL MÓDULO 11:

*"Profesor, aquí aplicamos una técnica clásica de diseño de pruebas: **Partición de Equivalencia**.*

*En el archivo `Ruc.php`, un RUC no es un número cualquiera de 11 dígitos. El último número es un dígito verificador que se calcula con una fórmula matemática llamada **Módulo 11**, multiplicando los números por los factores oficiales de SUNAT [5, 4, 3, 2, 7, 6, 5, 4, 3, 2].*

En `ValueObjectsTest.php`, creamos un 'Data Provider' con RUCs reales (como el del BCP o Alicorp) y con RUCs con números falsos o letras. El test confirma que el sistema rechace de inmediato cualquier RUC falso antes de que se guarde en la base de datos."

QUÉ DECIR EXPLICANDO LA CALCULADORA TRIBUTARIA (FACTURA MIXTA):

"Ahora pasamos a `CalculadoraTributariaTest.php`. En la prueba `testCalculaComprobanteMixtoComplejo()` evaluamos una situación que pasa mucho en un supermercado: en la misma compra llevas un monitor (que paga 18% de IGV), un libro (que está exonerado al 0%), un teclado de regalo y bolsas plásticas (que pagan 50 céntimos de impuesto ICBPER).

La prueba asegura que los impuestos no se mezclen: el 18% solo se aplica al monitor, y las bolsas pagan su impuesto aparte, dando el total exacto al centavo."

CONCEPTO CLAVE DE QA A RESALTAR:

Partición de Equivalencia y Valores Límite: Dividir las entradas en grupos válidos e inválidos para probar casos representativos sin tener que probar millones de combinaciones.

PASO 4 Pruebas de Integración: Seguridad e Idempotencia (2 minutos)

QUÉ CARPETA Y ARCHIVOS ABRIR EN VS CODE:

1. Carpeta: `backend/tests/Integration/` → Abre `XmlAndCryptoIntegrationTest.php` (líneas 92 a 119)
2. Carpeta: `backend/tests/Integration/` → Abre `SqlitePersistenceIntegrationTest.php` (líneas 87 a 118)

QUÉ DECIR SOBRE LA PRUEBA DE ADULTERACIÓN (SEGURIDAD CRIPTOGRÁFICA):

"En `XmlAndCryptoIntegrationTest.php` no probamos solo código PHP, sino seguridad real.

*Imaginen que una empresa emite una factura por 1,180 soles y alguien intenta hackear el archivo XML cambiando el texto de 1,180 a 10 soles para no pagar impuestos. En el método `testDetectaAdulteracionDeImporte...` simulamos exactamente ese ataque. Como el XML está firmado digitalmente con SHA-256, cualquier cambio de un solo carácter rompe el hash y el simulador de SUNAT lo rechaza con el **error fiscal 2335**."*

QUÉ DECIR SOBRE LA IDEMPOTENCIA (PREVENIR COBRO DOBLE):

> *"En `SqlitePersistenceIntegrationTest.php` probamos la **Idempotencia**.*
>
> *¿Qué pasa si la cajera presiona dos veces seguidas el botón 'Cobrar' por impaciencia, o si la conexión parpadea? Si no probamos esto, el sistema crearía dos facturas con números distintos y le cobraría doble al cliente. La prueba comprueba que si llega la misma solicitud, el sistema detecta que ya fue procesada y devuelve la misma factura original sin crear un duplicado."*

PASO 5 Pruebas de Casos de Uso con Mocks: Simulando a SUNAT (1.5 minutos)

QUÉ CARPETA Y ARCHIVO ABRIR EN VS CODE:

Carpeta: `backend/tests/Feature/` → Abre `FacturacionElectronicaMocksFeatureTest.php` (líneas 115 a 144)

QUÉ DECIR SOBRE LOS MOCKS Y EL ROLLBACK DE INVENTARIO:

*"Profesor, en QA se usa mucho el concepto de **Test Doubles o Mocks**. ¿Por qué usamos un Mock aquí?*

Porque no podemos estar llamando al servidor real de SUNAT en cada prueba: el servidor del estado se cae, tiene límites de conexión o requiere certificados de producción. Así que creamos un 'doble de prueba' que finge ser SUNAT.

*En el Caso 3, le decimos al Mock que responda con un rechazo fiscal: 'Contribuyente en Baja de Oficio' (código 2800). Aquí verificamos una regla de negocio crítica: teníamos 15 laptops en almacén y descontamos 10 para la venta. Al ver que SUNAT rechazó la factura, el sistema hace un **Rollback automático en el Kardex** y las 15 laptops regresan al stock. Si no probáramos esto, la tienda perdería inventario fantasma en el sistema."*

CONCEPTO CLAVE DE QA A RESALTAR:

Mocks vs Pruebas Reales: Los Mocks permiten probar situaciones hostiles o difíciles de provocar en la realidad (como caídas de red o rechazos tributarios raros) de manera controlada y repetible.

PASO 6 Demostración del Sistema Web en Funcionamiento (1.5 minutos)

QUÉ HACER EN EL NAVEGADOR WEB (HTTP://LOCALHOST:5173):

1. Muestra la pantalla del Facturador.
2. Haz clic en el botón **'Firmar & Emitir Factura Electrónica (UBL 2.1)'**.
3. Muestra el modal de confirmación: '*¡Comprobante emitido con éxito! F001-XXXX fue aceptado por SUNAT*'.
4. Haz clic en la pestaña **'Historial CPE'** y luego en el botón **'XML'** de una factura.
5. Finalmente, haz clic en la pestaña **'Auditoría PHPUnit 10.5'**.

QUÉ DECIR MIENTRAS OPERAS EL SISTEMA:

"Para cerrar, les muestro el sistema en funcionamiento en el navegador.

1. (Al emitir): Emitimos una factura para el Banco de Crédito. El sistema la firma digitalmente con la clave privada, la envía al Gateway y nos muestra el modal de confirmación con el código CDR 0 de SUNAT.

2. (En el Historial): Si abrimos el XML firmado, podemos ver el estándar UBL 2.1 oficial de SUNAT con el bloque `<ds:Signature>` que acabamos de validar en los tests.

3. (En la pestaña Auditoría): Y para facilitar las cosas a los auditores o docentes, integramos la consola de PHPUnit directamente en la interfaz web, de modo que se pueden correr las pruebas y ver los resultados en tiempo real sin abrir una terminal negra."

Preguntas Típicas que te Puede Hacer el Profesor y Cómo Responderlas

Prepárate con estas respuestas breves, claras y certeras. Si el profesor te interrumpe o te hace preguntas técnicas, responde con esta seguridad:

1. ¿Por qué decidieron usar SQLite en memoria (:memory:) en lugar de MySQL o PostgreSQL para los tests?

Respuesta: "Por velocidad y aislamiento. Una base de datos MySQL en disco tarda varios segundos en conectar y deja datos residuales después de cada prueba. Con SQLite en memoria, la base de datos se crea vacía en milisegundos en la memoria RAM y se destruye al terminar el test, asegurando que ninguna prueba contamine a la siguiente y que la suite corra en solo 0.2 segundos."

2. ¿Cuál es la diferencia entre una prueba Unitaria y una prueba de Integración en tu proyecto?

Respuesta: "La prueba unitaria evalúa una clase o función aislada sin interactuar con nada externo, como la `CalculadoraTributaria` calculando el 18% del IGV. La prueba de integración, en cambio, comprueba que dos componentes distintos trabajen bien juntos, como el generador de XML conectándose con la librería de firma digital o el repositorio guardando datos reales en SQLite."

3. ¿Qué es un Mock y por qué no llamaron directamente a los servidores web de SUNAT?

Respuesta: "Un Mock es un objeto simulado que imita el comportamiento de un servicio externo. No llamamos a SUNAT directamente en los tests porque los servidores de SUNAT son lentos, requieren internet, a veces están en mantenimiento y tienen límites de tráfico. Con el Mock podemos simular respuestas inmediatas y forzar escenarios difíciles, como caídas de red o rechazos fiscales, en un entorno 100% controlado."

4. ¿Qué técnica de diseño de pruebas usaste para los límites de dinero?

Respuesta: "Usamos *Análisis de Valores Límite*. Por ejemplo, la ley de SUNAT dice que las boletas anónimas a 'Clientes Varios' solo se permiten hasta S/ 700.00 soles. Probamos exactamente en S/ 700.00 (donde sí lo permite) y en S/ 700.01 o superior (donde el sistema lanza la excepción Error 2511 exigiendo DNI). Los errores siempre ocurren en los bordes de las condiciones."

5. ¿Qué significa el código CDR 0 que devuelve SUNAT?

Respuesta: "CDR significa 'Constancia de Recepción'. El código 0 es el código de éxito oficial de SUNAT que indica que el comprobante UBL 2.1 fue validado, la firma digital fue comprobada y la factura fue aceptada legalmente en el sistema tributario peruano."